

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

October 9, 2025

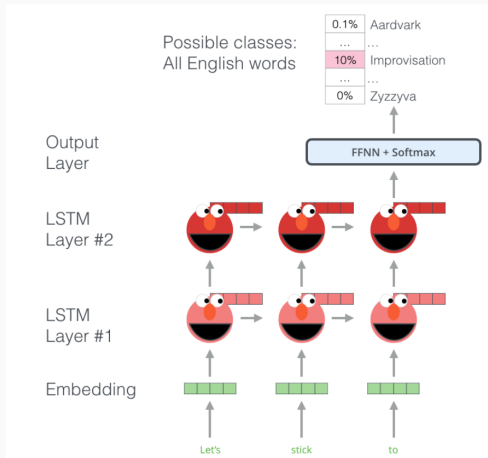
Lecture 13

Run the code

- To train the model, run: `python main.py`
- To train the model with specific arguments, run: `python main.py --batch_size=64` . The following command-line arguments are available:
 - Batch size: `--batch_size`
 - bptt: `--bptt`
 - Learning rate: `--lr`
 - Embedding size: `--embed_size`
 - Hidden size: `--hidden_size`
 - Latent size: `--latent_size`

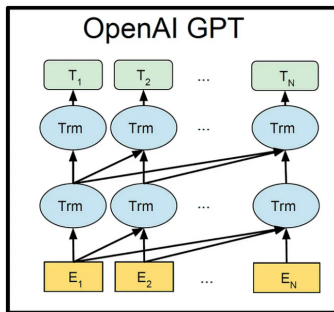
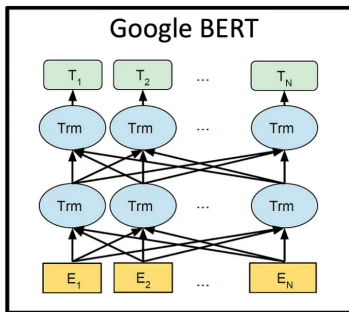
Language model pretraining task

- Predict the next word given the prefix
- Can be defined on any unlabeled document



GPT models

- BERT: pretraining by masked language models (Transformer encoder)
- GPT: directly train an autoregressive language model: (Transformer decoder) learns to generate text from left to right
- Better ability for text generation

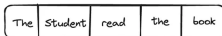


Encoder-Decoder, Encoder-Only, Decoder-Only

Masked Language Modeling (MLM)



objective: Reconstruct word



Bidirectional

Autoregressive (Causal)
Language Modeling



objective: Predict next word



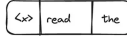
Unidirectional

Span Corruption



sentinel token

objective: Reconstruct span



Bidirectional

Causal Attention

```
def forward(self, x):
    B, T, C = x.size() # batch size, sequence length, embedding dimensionality (n_embd)

    # calculate query, key, values for all heads in batch and move head forward to be the batch dim
    q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
    k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
    q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
    v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)

    # causal self-attention; Self-attend: (B, nh, T, hs) x (B, nh, hs, T) -> (B, nh, T, T)
    att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
    att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float('-inf'))
    att = F.softmax(att, dim=-1)
    att = self.attn_dropout(att)
    y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)
    y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head outputs side by side

    # output projection
    y = self.resid_dropout(self.c_proj(y))
    return y
```

Causal Attention

```
b.attn.bias[:, :, :T, :T]
```

```
tensor([[[[1., 0., 0., 0., 0., 0.],  
          [1., 1., 0., 0., 0., 0.],  
          [1., 1., 1., 0., 0., 0.],  
          [1., 1., 1., 1., 0., 0.],  
          [1., 1., 1., 1., 1., 0.],  
          [1., 1., 1., 1., 1., 1.]]]])
```

mask fill

```
def forward(self, x):
    B, T, C = x.size() # batch size, sequence length, embedding dimensionality (n_embd)

    # calculate query, key, values for all heads in batch and move head forward to be the batch dim
    q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
    k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
    q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
    v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)

    # causal self-attention; Self-attend: (B, nh, T, hs) x (B, nh, hs, T) -> (B, nh, T, T)
    att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
    att = att.masked_fill(self.bias[:, :, T, T] == 0, float('-inf'))
    att = F.softmax(att, dim=-1)
    att = self.attn_dropout(att)
    y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)
    y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head outputs side by side

    # output projection
    y = self.resid_dropout(self.c_proj(y))
    return y
```

why convert 0 to -inf?

```
def forward(self, x):
    B, T, C = x.size() # batch size, sequence length, embedding dimensionality (n_embd)

    # calculate query, key, values for all heads in batch and move head forward to be the batch dim
    q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
    k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
    q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
    v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)

    # causal self-attention; Self-attend: (B, nh, T, hs) x (B, nh, hs, T) -> (B, nh, T, T)
    att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
    att = att.masked_fill(self.bias[:, :, T, T] == 0, float('-inf'))
    att = F.softmax(att, dim=-1)
    att = self.attn_dropout(att)
    y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)
    y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head outputs side by side

    # output projection
    y = self.resid_dropout(self.c_proj(y))
    return y
```

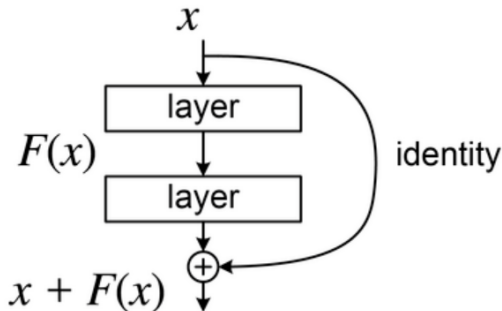
Block: What is block doing?

```
class Block(nn.Module):
    """ an unassuming Transformer block """

    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = nn.ModuleDict(dict(
            c_fc    = nn.Linear(config.n_embd, 4 * config.n_embd),
            c_proj  = nn.Linear(4 * config.n_embd, config.n_embd),
            act      = NewGELU(),
            dropout  = nn.Dropout(config.resid_pdrop),
        ))
        m = self.mlp
        self.mlpf = lambda x: m.dropout(m.c_proj(m.act(m.c_fc(x)))) # MLP forward

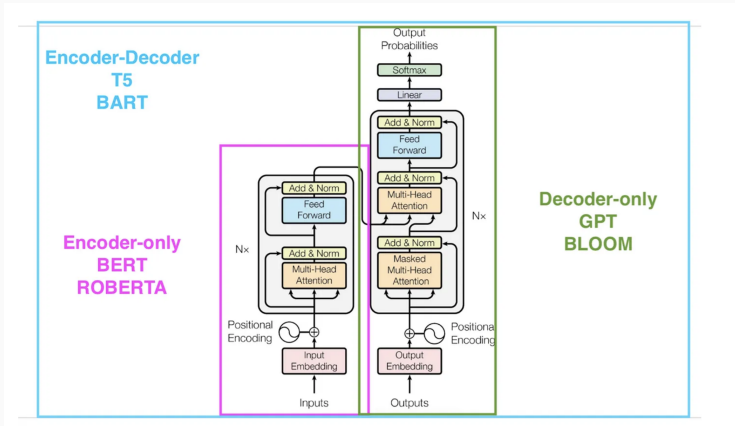
    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlpf(self.ln_2(x))
        return x
```

Residual Neural Network



A residual block in a deep residual network. Here, the residual connection skips two layers.

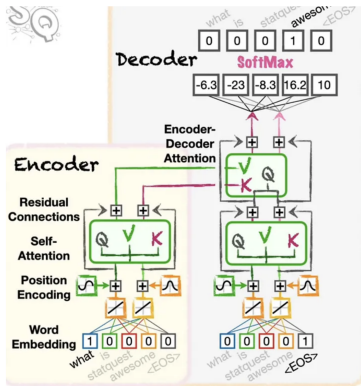
Encoder-Decoder, Encoder-Only, Decoder-Only



<https://medium.com/@reyhaneh.esmailbeigi/bert-gpt-and-bart-a-short-comparison-5d6a57175fca>

<https://medium.com/@row3no6/why-chatgpt-uses-decoder-only-eaf0223143e6>

Encoder-Decoder, Encoder-Only, Decoder-Only

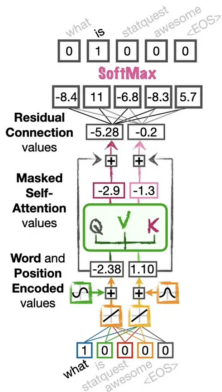


A normal **Transformer** uses one unit to encode the input, called the **Encoder**, and a separate unit to generate the output, called the **Decoder**.

And a normal **Transformer** uses two types of **Attention** during inference: **Self-Attention** and **Encoder-Decoder Attention**.

Lastly, during **Training**, a normal **Transformer** uses **Masked Self-Attention**, but only on the output.

Encoder-Decoder, Encoder-Only, Decoder-Only



In contrast, a **Decoder-Only Transformer** has a single unit for both encoding the input and generating the output.

And a **Decoder-Only Transformer** uses a single type of attention, **Masked Self-Attention**.

And a **Decoder-Only Transformer** uses **Masked Self-Attention** all the time on everything, the input and the output.

Encoder-Decoder, Encoder-Only, Decoder-Only

	BERT	GPT	BART
Model Type	Encoder Only	Decoder Only	Encoder-Decoder
Direction	Bidirectional	Unidirectional (left-to-right)	Bidirectional
Pre-training Objective	Masked language modeling (MLM)	Autoregressive (casual) language modeling	Span Corruption (Masking entire spans of words)
Fine-tuning	Task-specific layer added on top of the pre-trained BERT model	Providing task-specific prompts using few-shot or one-shot adaptation and adapting the model's parameters	Versatile and can be used for various NLP tasks
Use Case	Sentiment Analysis Named entity Recognition Word Classification	Text generation Text completion creative writing	Translation Text Summarisation Question & Answer
Original Organisations	Google AI	OpenAI	Facebook AI

- GPT → GPT-2
- Model size ↑, training data size ↑



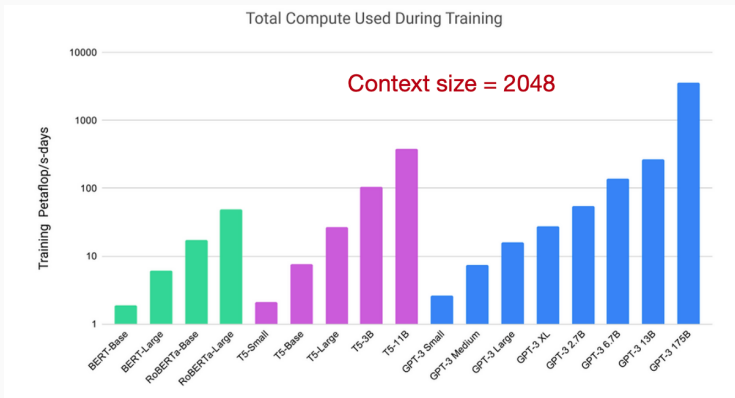
Context size = 1024



.. trained on 40Gb of Internet text ..

(Radford et al., 2019): Language Models are Unsupervised Multitask Learners

- GPT-2 $\rightarrow$ GPT-3:
- Larger model 1.5B $\rightarrow$ 175B (number of parameters)
- Bigger training set 14B tokens $\rightarrow$ 300B tokens



GPT-3: In-context learning

- Language models can learn from a few examples (few-shot learner) without changing the model
- Idea: put the examples as the prefix of the language models (prompt)
- Just forward pass, no need to fine-tune the model

Circulation revenue has increased by 5% in Finland. // Positive

Panostaja did not disclose the purchase price. // Neutral

Paying off the national debt will be extremely painful. // Negative

The company anticipated its operating profit to improve. // _____



Circulation revenue has increased by 5% in Finland. // Finance

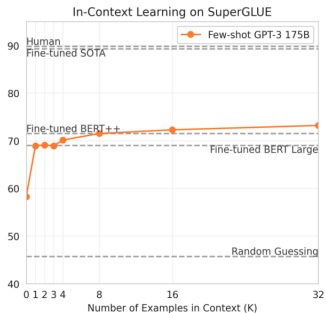
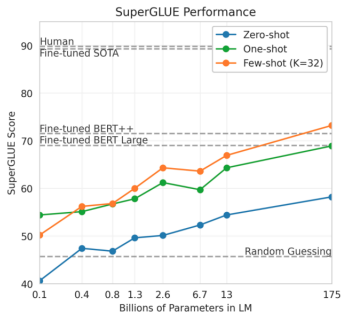
They defeated ... in the NFC Championship Game. // Sports

Apple ... development of in-house chips. // Tech

The company anticipated its operating profit to improve. // _____



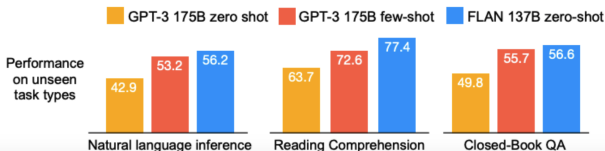
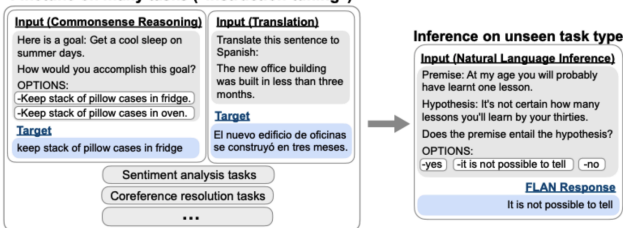
Few-shot performance v.s. fine-tuning



Basic Instruction Tuning

(Wei et al. 2021, Sanh et al. 2021)

Finetune on many tasks (“instruction-tuning”)



Instruction Tuning Datasets

- Good reference: FLAN Collection (Longpre et al. 2023)

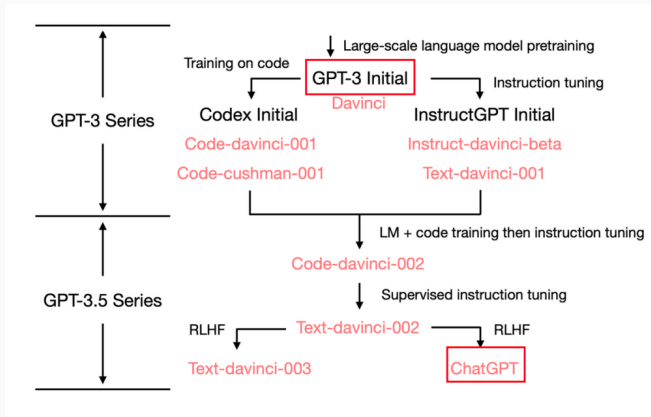
Release	Collection	Model Details			Data Collection & Training Details				Methods
		Model	Base	Size	Public?	Prompt Types	Tasks in Flan	# Exs	
2020 05	UnifiedQA	UnifiedQA	RoBerta	110-340M	P	ZS	46 / 46	750k	
2021 04	CrossFit	BART-CrossFit	BART	140M	NP	FS	115 / 159	71M	
2021 04	Natural Inst v1.0	Gen. BART	BART	140M	NP	ZS / FS	61 / 61	620k	+ Detailed k-shot Prompts
2021 09	Flan 2021	Flan-LaMDA	LaMDA	137B	NP	ZS / FS	62 / 62	4.4M	+ Template Variety
2021 10	P3	T0, T0+, T0++	T5-LM	3-11B	P	ZS	62 / 62	12M	+ Template Variety + Input Inversion
2021 10	MetalCL	MetalCL	GPT-2	770M	P	FS	100 / 142	3.5M	+ Input Inversion + Noisy Channel Opt
2021 11	ExMix	ExT5	T5	220M-11B	NP	ZS	72 / 107	500k	+ With Pretraining
2022 04	Super-Natural Inst.	Tk-Instruct	T5-LM, mT5	11-13B	P	ZS / FS	1556 / 1613	5M	+ Detailed k-shot Prompts + Multilingual
2022 10	GLM	GLM-130B	GLM	130B	P	FS	65 / 77	12M	+ With Pretraining + Bilingual (en, zh-cn)
2022 11	xP3	BLOOMz, mT0	BLOOM, mT5	13-176B	P	ZS	53 / 71	81M	+ Massively Multilingual
2022 12	Unnatural Inst. [†]	T5-LM-Unnat. Inst.	T5-LM	11B	NP	ZS	~20 / 117	64k	+ Synthetic Data
2022 12	Self-Instruct [†]	GPT-3 Self Inst.	GPT-3	175B	NP	ZS	Unknown	82k	+ Synthetic Data + Knowledge Distillation
2022 12	OPT-IML Bench [†]	OPT-IML	OPT	30-175B	P	ZS + FS CoT	~2067 / 2207	18M	+ Template Variety + Input Inversion + Multilingual
2022 10	Flan 2022 (ours)	Flan-T5, Flan-PaLM	T5-LM, PaLM	10M-540B	P NP	ZS + FS CoT	1836	15M	+ Template Variety + Input Inversion + Multilingual

Instruction Tuned Models

- **FLAN-T5:** huggingface/google/flan-t5-xxl
 - Encoder-decoder model based on T5
 - 11B parameters
- **LLaMa-2 Chat:** huggingface/meta-llama/Llama-2-70b-chat-hf
 - Decoder-only model
 - 70B parameters
- **Mixtral instruct:** huggingface/mistralai/Mixtral-8x7B-Instruct-v0.1
 - Decoder-only mixture of experts model
 - 45B parameters
- *(smaller versions also available - Mistral, LLaMa2-7B)*

What's new in ChatGPT

- Training on code
- Supervised instruction tuning
- Reinforcement learning from human feedback (RLHF)



Supervised instructional tuning

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.



Explain reinforcement learning to a 6 year old.

A labeler demonstrates the desired output behavior.



We give treats and punishments to teach...

This data is used to fine-tune GPT-3.5 with supervised learning.



SFT



Step 2

Collect comparison data and train a reward model.

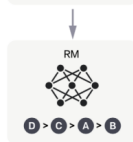
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.

